



Iby and Aladar Fleischman
Faculty of Engineering
Tel Aviv University

הפקולטה להנדסה
ע"ש איבי ואלדר פליישימן
אוניברסיטת תל-אביב

Spline Interpolation DSP ASIC: Full RTL-to-Digital Design (TSMC 65nm)

Final Project Report

Project Number: 3439

Student: Maria Najjar - ID 325040350

Student: Malak Khaskia - ID 211962410

Supervisor: Mohammad Khamaysi

Project carried out at: Tel Aviv University, Faculty of Engineering

Contents

Abstract.....	4
1 Introduction	5
1.1 Motivation.....	5
1.2 Project Goals	5
1.3 Approach to Solving the Problem	5
1.4 Algorithm Comparison	6
2 Theoretical Background	7
2.1 QAM Signals and I/Q Representation	7
2.2 Multi-Rate Interpolation	7
2.3 Cubic Hermite interpolation and MINAJ2	8
2.3.1 MINAJ2 Slope Estimation	8
2.4 Programmable FIR filtering	9
2.5. Frequency-Domain Analysis	9
2.6 Fixed-Point Representation	9
2.7 Error Vector Magnitude (EVM)	10
3 Simulation and MATLAB Model	11
3.1 Input and Reference-Signal Generation.....	11
3.2 Interpolation-Algorithm Evaluation and MINAJ2 Model	11
3.3 Fixed-Point Conversion	12
3.4 FIR-Filter Development	13
3.5 Final MATLAB Results.....	13
4 ASIC Implementation	15
4.1 Design flow and tools.....	15
4.2 System Requirements and Architecture	15
4.3 RTL Architecture and Implementation.....	16
4.4 Verification.....	17
4.4.1 Functional Verification	17
4.4.2 Formal Verification.....	17
4.4.3 Coverage	17
4.5 Synthesis	18
4.5.1 Timing Constraints	18
4.5.2 Logic Optimization and Clock Gating	18
4.5.3 Synthesis Results	18
4.6 Post-Synthesis Validation.....	18

4.6.1 Logic Equivalence Checking.....	18
4.6.2 Pad Integration and GLS.....	18
5 Physical Design	19
5.1 Design Initialization	19
5.2 Floorplanning and Power Grid	19
5.3 Placement and Clock Tree Synthesis.....	19
5.4 Routing and Design Closure	19
5.5 Parasitic Extraction.....	19
5.6 Post-Layout Static Timing Analysis.....	20
5.7 Post-Route Power Analysis	20
5.8 Physical Verification	20
6 Results and Analysis	21
6.1. RTL vs. MATLAB EVM	21
6.2 Verification Results	21
6.2.1 Functional Simulation Results	21
6.2.2 Coverage Results	22
6.2.3 Formal Verification Results	22
6.3 Physical-Design Results	23
6.4 Post-Route Power Results.....	24
6.5 IR Drop	24
6.6 PrimeTime Timing Sign-Off	24
6.7 DRC and LVS	25
6.8 Final Sign-Off Summary.....	26
7. Summary, Conclusions, and Future Work.....	27
7.1 Evaluation Against the Project Objectives	27
7.2 Conclusions	27
7.3 Suggested Performance Improvements	28
7.4 Future Work	28
8 Project Documentation.....	29
References	30

Abstract

This project presents the design, verification, and physical implementation of a real-time ASIC for spline-based interpolation of complex baseband I/Q signals. The system processes 64-QAM samples at an input rate of 60 MSa/s and supports programmable interpolation factors producing output sampling rates of 120, 180, 240, and 300 MSa/s. The objective is to increase the sampling rate while preserving I/Q synchronization and maintaining the required signal quality.

The processing chain consists of the MINAJ2 cubic interpolation algorithm followed by a configurable 10-tap FIR low-pass filter for suppression of unwanted spectral components. The hardware architecture contains two parallel I and Q processing paths controlled by shared configuration and timing logic. During initialization, the interpolation factor and ten 16-bit FIR coefficients are loaded serially before streaming begins. Depending on the selected interpolation factor, the chip operates with either a 960 MHz clock and 16-bit input words or a 900 MHz clock and 15-bit input words, while maintaining the same 60 MSa/s input sampling rate.

The project covers the complete ASIC development flow, including MATLAB floating- and fixed-point modeling, synthesizable RTL implementation, functional and formal verification, logic synthesis, gate-level verification, physical design, parasitic extraction, power analysis, and final timing sign-off in TSMC 65 nm technology. Comparison of the RTL output with the MATLAB reference model produced EVM values between approximately 72.8 and 186.4 m%rms for the supported interpolation factors, satisfying the required maximum of 350 m%rms. Final Synopsys Primetime analysis showed no setup or hold violations, with worst reported slacks of +0.29 ns and +0.86 ns, respectively. The implemented chip occupies a 0.950 mm² die with a 0.385 mm² core and approximately 138k equivalent gates, demonstrating the successful transition from algorithm-level development to a complete post-layout ASIC implementation.

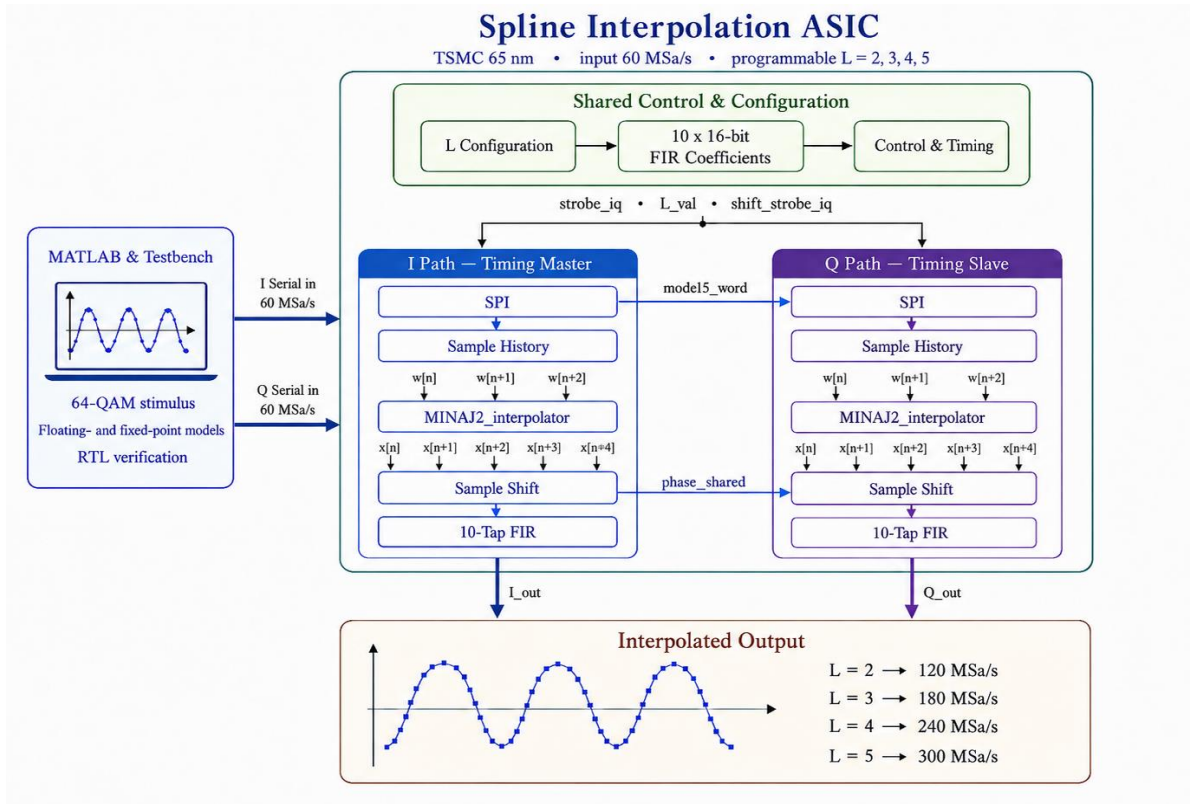


Figure 1: High-Level Overview of the Programmable Spline-Interpolation ASIC and Its Signal-Processing Chain

1 Introduction

1.1 Motivation

Increasing the sampling rate of a digital baseband signal is an essential operation in many communication transmitters. For complex QAM signals, the interpolation process must generate additional I/Q samples without significantly changing the waveform or degrading the transmitted constellation.

The challenge becomes greater when the interpolation is implemented as a dedicated ASIC. The selected method must not only achieve good reconstruction accuracy but also translate efficiently into fixed-point arithmetic and sustain high real-time data rates. This creates a trade-off between signal quality and implementation cost, since more accurate interpolation methods may require additional samples, arithmetic operations, wider data paths, or more complex control.

The project therefore focuses on developing a configurable interpolation architecture that achieves the required signal quality while remaining practical for high-speed hardware implementation.

1.2 Project Goals

The project aimed to develop a complete ASIC solution for multi-rate interpolation of complex baseband I/Q signals.

The main objectives were to:

- evaluate different interpolation algorithms and determine their suitability for hardware *implementation*;
- develop floating-point MATLAB models of the candidate methods in order to evaluate their interpolation accuracy before introducing quantization effects.
- convert the selected solution to a fixed-point representation suitable for digital hardware.
- Receive complex I/Q samples at a fixed input rate of 60 MSa/s.
- Support programmable interpolation factors $L \in \{2, 3, 4, 5\}$.
- preserve synchronization between the I and Q processing channels;
- Apply programmable FIR filtering to suppress spectral images.
- Meet the project-defined signal-quality limit of $\text{EVM} \leq 350\text{m\% RMS}$.
- Complete an RTL-to-GDSII flow including synthesis, STA, place and route, LEC, GLS, power, and physical verification.

1.3 Approach to Solving the Problem

The project began at the algorithmic level. Several candidate interpolation methods were implemented in MATLAB and tested using the same 64-QAM signal environment. The comparison considered reconstruction performance measured using EVM, together with

characteristics that directly affect hardware implementation, such as the number of neighboring samples required, arithmetic complexity, and sensitivity to finite word length.

Following this study, MINAJ2 was selected for the final design. A fixed-point MATLAB implementation was then developed to define the numerical representation used in hardware and to provide a reference for RTL verification.

The hardware architecture was subsequently implemented in SystemVerilog. It consists of shared configuration and timing logic together with two parallel datapaths for the I and Q components. Each Datapath performs MINAJ2 interpolation followed by a programmable symmetric 10-tap FIR filter.

The design was verified against the MATLAB golden model and then implemented through logic synthesis, physical design, logical equivalence checking, gate-level simulation, and power analysis.

1.4 Algorithm Comparison

Several interpolation algorithms were evaluated before selecting the method used in the final ASIC. Linear interpolation offered the lowest hardware complexity but did not provide sufficient reconstruction accuracy. Catmull–Rom interpolation produced smoother results but required a larger sample neighborhood and could introduce overshoot. Cubic B-spline interpolation provided high smoothness but involved greater implementation complexity and did not necessarily pass directly through the original samples. MINAJ2 provided the best balance between EVM performance, local sample requirements, fixed-point behavior, and hardware complexity. It was therefore selected for the final implementation.

2 Theoretical Background

2.1 QAM Signals and I/Q Representation

Quadrature Amplitude Modulation (QAM) transmits information by varying the amplitudes of two orthogonal components separated by 90° . These components are the in-phase component I and the quadrature component Q . The discrete-time complex baseband signal is represented as $s[n]=I[n]+jQ[n]$, where $I[n]$ is the real component and $Q[n]$ is the imaginary component.

Each QAM symbol corresponds to a point (I,Q) in the constellation plane. This project uses 64-QAM, which contains 64 possible constellation points; therefore, each symbol represents $\log_2(64)=6$ bits of information.

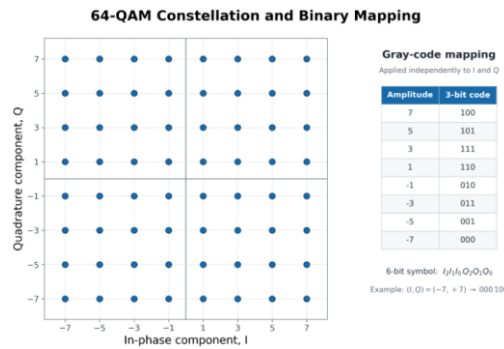


Figure 2: Ideal 64-QAM Constellation.

The MATLAB input signal is pulse-shaped using a Root Raised Cosine (RRC) filter with a roll-off factor of $\alpha=0.15$.

Pulse shaping limits the occupied bandwidth and reduces inter symbol interference. The resulting I and Q sample streams are processed by two parallel paths that perform the same interpolation and filtering

2.2 Multi-Rate Interpolation

The sampling rate determines the number of signal samples processed per second. Increasing this rate is required in applications such as oversampling, multirate signal processing and preparation for digital-to-analog conversion.

For an input sampling rate F_{in} and interpolation factor L , the output sampling rate is $F_{out}=L \cdot F_{in}$.

In this project, $F_{in}=60$ MSa/s, and the supported factors $L \in \{2,3,4,5\}$ produce output rates of

120, 180, 240, and 300 MSa/s, respectively.

Increasing the sampling rate requires generating new samples between consecutive input samples. Since these values are not directly available, they are estimated using an interpolation algorithm. The algorithm selected for this project is MINAJ2 cubic interpolation.

The sampling-rate increase also produces unwanted spectral images outside the desired baseband. Therefore, the interpolated signal is passed through a low-pass FIR filter that attenuates these images while preserving the required baseband spectrum.

2.3 Cubic Hermite interpolation and MINAJ2

MINAJ2 constructs the interpolated signal using cubic Hermite segments. Consider two consecutive samples $x[i]$ and $x[i+1]$, with corresponding normalized slopes m_i and m_{i+1} .

The normalized position within the interval is $u = \frac{r}{L}$ $r = 0, 1, 2, \dots, L-1$

The interpolated value is:

$$(1) S_i(u) = H_{00}(u)x_i + H_{01}(u)x_{i+1} + H_{10}(u)m_i + H_{11}(u)m_{i+1} \quad u \in [0, 1]$$

The Hermite basis functions are:

$$H_{00}(u) = 2u^3 - 3u^2 + 1 \quad H_{01}(u) = -2u^3 + 3u^2$$

$$H_{10}(u) = u^3 - 2u^2 + u \quad H_{11}(u) = u^3 - u^2$$

Once the endpoint values and slopes are known, intermediate samples can be generated by evaluating the polynomial at the required fractional positions.

2.3.1 MINAJ2 Slope Estimation

The principal difference between cubic Hermite interpolation methods is the method used to estimate the slopes. MINAJ2 is a local, look-ahead heuristic derived by minimizing the accumulated squared jerk over two consecutive candidate spline segments:

$$(2) \min J(m_i) = \int_{t_{i-1}}^{t_i} (S_{i-1}'''(t))^2 dt + \int_{t_i}^{t_{i+1}} (S_i'''(t))^2 dt$$

Instead of solving a global optimization problem over the complete signal, MINAJ2 performs the calculation locally using neighboring samples.

For uniformly spaced samples, the resulting recursive slope expression is

$$(3) m_i = \frac{-11x_{i-1} - 4m_{i-1} + 8x_i + 3x_{i+1}}{10}$$

Therefore, calculation of the current slope requires only three local samples, the previously calculated slope, and constant coefficients. This avoids the global matrix solution required by classical spline methods and makes MINAJ2 suitable for streaming, fixed-point, and RTL implementations.

After the required slopes are calculated, they are substituted into the Hermite interpolation equation to generate the intermediate samples.

The hardware value of u is restricted to the discrete phases $u = r/L$ for $r = 0, \dots, L-1$. This converts a continuously variable polynomial into a small set of phase-dependent constants and operations.

2.4 Programmable FIR filtering

Increasing the sampling rate produces spectral images outside the desired baseband. A programmable low-pass FIR filter is therefore applied after interpolation to preserve the required baseband spectrum and attenuate these unwanted images.

The FIR output is:

$$(4) y[n] = \sum_{k=0}^{N-1} h[k]x[n-k]$$

where $x[n]$ is the interpolated input, $h[k]$ is the k -th filter coefficient, and $y[n]$ is the filtered output.

The implemented filter contains $N=10$ taps. All ten coefficients are loaded serially, and the same coefficient set is used by the I-channel and Q-channel filters.

2.5. Frequency-Domain Analysis

The Fast Fourier Transform (FFT) is an efficient algorithm for computing the Discrete Fourier Transform (DFT), which represents a discrete-time signal in the frequency domain. For a sequence $x[n]$ of length N , the DFT is

$$(5) X[k] = \sum_{n=0}^{N-1} x[n] e^{-\frac{j2\pi kn}{N}}$$

While the time-domain waveform shows how the signal changes from sample to sample, the FFT reveals how the signal energy is distributed over frequency.

In this project, FFT-based analysis was used to evaluate the spectral behavior of the interpolation process. The spectra of the reference, floating-point, and fixed-point signals were compared to verify that the desired baseband spectrum was preserved and to identify unwanted spectral images introduced by the increase in sampling rate.

The FFT was also used to evaluate the effect of the FIR low-pass filter. By comparing the signal spectrum before and after filtering, the attenuation of unwanted out-of-band components could be observed.

2.6 Fixed-Point Representation

Floating-point models provide a wide numerical range and high precision, but floating-point arithmetic generally requires greater hardware area and power. Therefore, the RTL implementation uses a fixed-point representation.

In fixed-point arithmetic, a predefined number of bits is assigned to the integer and fractional parts. A value represented with F fractional bits is interpreted according to

$$(6) x_{real} = \frac{x_{integer}}{2^F}$$

In the MATLAB model, the signal samples are represented using 16 bits with 12 fractional bits. The interpolation coefficients require a wider numerical range, since some of them reach values between approximately -2.5 and 2.5 . They are therefore represented using 16 bits with 12 fractional bits.

This representation provides sufficient precision for the interpolation while keeping the arithmetic suitable for efficient hardware implementation.

2.7 Error Vector Magnitude (EVM)

Error Vector Magnitude (EVM) is used as the primary quantitative metric for evaluating the interpolation algorithms and the final hardware implementation. In a 64-QAM constellation, each transmitted symbol corresponds to one of 64 ideal points in the I/Q plane. Interpolation errors, coefficient quantization, finite-word-length effects, and filtering may displace a recovered symbol from its ideal constellation point.

For the k -th symbol, the error vector is the difference between the measured and ideal constellation points:

$$(7) e[n] = X[n] - X_{ref}[n] = (I[n] - I_{ref}[n]) + j(Q[n] - Q_{ref}[n])$$

.

The RMS EVM is calculated as

$$(8) EVM_{rms}(\%) = 100 \sqrt{\frac{\sum_{n=0}^{N-1} |e[n]|^2}{\sum_{n=0}^{N-1} |x_{ref}[n]|^2}}$$

EVM therefore measures the average normalized displacement of the recovered symbols from their ideal 64-QAM constellation points. A lower EVM indicates that the reconstructed constellation is closer to the ideal constellation. The project requirement is:

$$EVM_{rms} \leq 350m_{rms} = 0.35\%_{rms}$$

EVM was used during algorithm selection, fixed-point evaluation, and verification of the final hardware implementation.

3 Simulation and MATLAB Model

MATLAB was used to generate the input signal, select the interpolation algorithm, develop its floating-point and fixed-point models, and evaluate the FIR filter. The complete model was validated using waveform comparisons, spectral analysis, EVM, and 64-QAM constellation diagrams.

3.1 Input and Reference-Signal Generation

The test signal was generated using the IQTools iqmod QAM generator. It consisted of 4000 random 64-QAM symbols shaped by a Root Raised Cosine filter. A fixed random seed was used so that the same signal was produced in every simulation run.

The generated complex baseband signal is $s_{in}[n] = I_{in}[n] + jQ_{in}[n]$,

where $I_{in}[n]$ and $Q_{in}[n]$ are the signals applied to the two chip inputs. The QAM mapping and pulse shaping are performed outside the ASIC.

Table 1: MATLAB input-signal and processing configuration.

Parameter	Value	Report notes
Modulation	64-QAM	Complex baseband I/Q signal
Input sampling rate	60 MSa/s	Common to all operating modes
Signal bandwidth	20 MHz	Use the same bandwidth definition in all MATLAB plots
Number of symbols	4000	A fixed random seed ensures repeatable results
Oversampling factor	3	Used by the IQTools signal generator
Pulse-shaping filter	Root Raised Cosine	Filter span of 80 symbols
RRC roll-off factor	$\alpha=0.15$	Input and reference signals are normalized before processing
Interpolation factors	$L \in \{2,3,4,5\}$	Output rate is 60L MSa/s
Signal normalization	$s[n]/(2\text{RMS}\{s[n]\})$	

For each interpolation factor, the same QAM sequence was also generated directly at the corresponding output sampling rate. This high-rate signal was used as the reference for evaluating interpolation accuracy.

Two reference levels were used during development: the directly generated high-rate QAM signal was used to evaluate algorithmic reconstruction accuracy, while the floating-point MINAJ2 model was later used as the golden reference for the fixed-point and RTL implementations.

3.2 Interpolation-Algorithm Evaluation and MINAJ2 Model

The interpolation algorithm was not predetermined at the beginning of the project. Several interpolation methods were evaluated, including Catmull-Rom, B-spline, MATLAB cubic-spline

interpolation, and MINAJ2. The comparison considered reconstruction accuracy together with arithmetic complexity, required sample storage, and suitability for continuous hardware processing.

MINAJ2 was selected because it provided a suitable balance between interpolation quality and hardware complexity. Its computation depends only on neighboring samples and a previously calculated slope, which makes the method suitable for a streaming implementation.

The selected algorithm was first implemented using floating-point arithmetic in MATLAB. The model implemented the Hermite interpolation and MINAJ2 slope equations introduced in Section 2. Its output was aligned with the directly generated high-rate reference using cross-correlation before calculating the reconstruction error.

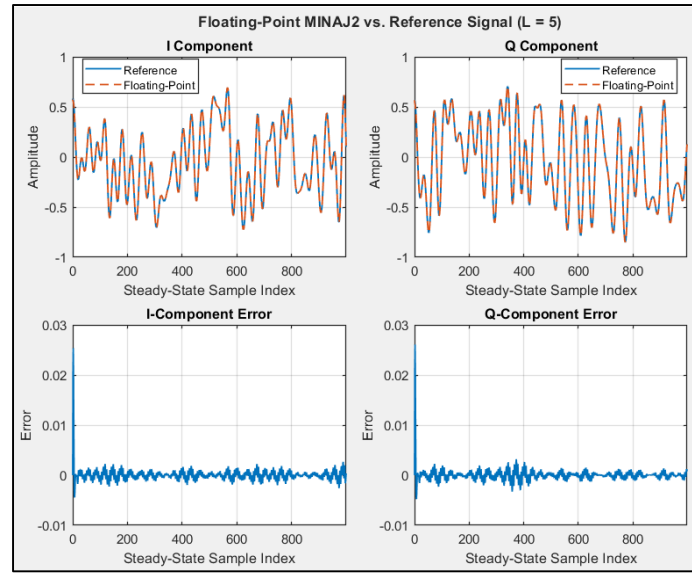


Figure 3: Floating-point MINAJ2 and reference signal comparison for L=5

3.3 Fixed-Point Conversion

After validating the floating-point algorithm, the MINAJ2 model was converted to fixed-point arithmetic to represent the numerical behavior expected in hardware.

The final MINAJ2 datapath uses a signed 16-bit representation with 12 fractional bits. Intermediate calculations use sufficient precision for multiplication and accumulation before conversion to the final output format. Rounding and overflow behavior were explicitly modeled in MATLAB to reproduce the arithmetic used by the hardware implementation.

Fixed-point configuration: signed representation, WL=16, FL=12, nearest rounding, and saturation overflow handling.

The fixed-point output was compared with both the floating-point MINAJ2 model and the high-rate reference. Any remaining difference is caused by coefficient quantization, rounding, and finite arithmetic precision.

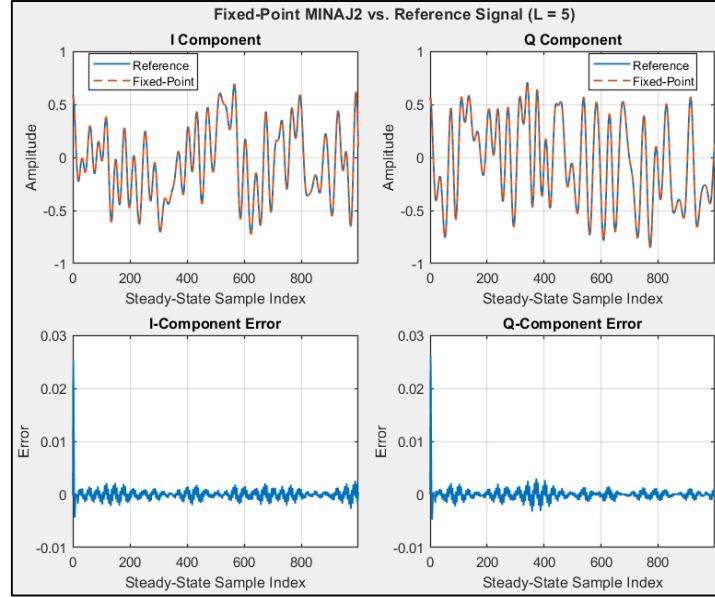


Figure 4: Fixed-Point MINAJ2 and Reference-Signal Comparison for L=5

The floating-point and fixed-point outputs closely follow the reference signal. The small difference between them results from coefficient quantization, rounding, and finite arithmetic precision.

3.4 FIR-Filter Development

A programmable 10-tap FIR low-pass filter was placed after the MINAJ2 interpolation stage to suppress the spectral images introduced by up-sampling while preserving the useful signal band. The filter coefficients were designed in MATLAB using the `firpm` function, which implements the Parks–McClellan equiripple optimization method. For each interpolation factor ($L \in \{2, 3, 4, 5\}$), the filter was designed according to the corresponding output sampling frequency, ($f_{s_{out}} = L * 60 \text{ MSa/s}$), using a 14 MHz passband, a 44 MHz stopband, and the weights [10000, 200]. The resulting coefficients were evaluated in floating point and then quantized to signed 16-bit values with 12 fractional bits for use in both the MATLAB fixed-point model and the RTL implementation. The same set of ten coefficients was applied to the I and Q channels, and the filter performance was verified by comparing the signal spectra before and after filtering using identical FFT and normalization settings.

3.5 Final MATLAB Results

The complete MATLAB model was evaluated for all supported interpolation factors, $L \in \{2, 3, 4, 5\}$.

Before EVM evaluation, initialization transients were excluded and the reconstructed signals were aligned with the reference, so that only steady-state samples were analyzed.

The final reconstructed signals were also evaluated using the Vector Signal Analysis environment (VSA) with the same 64-QAM and RRC configuration used for signal generation.

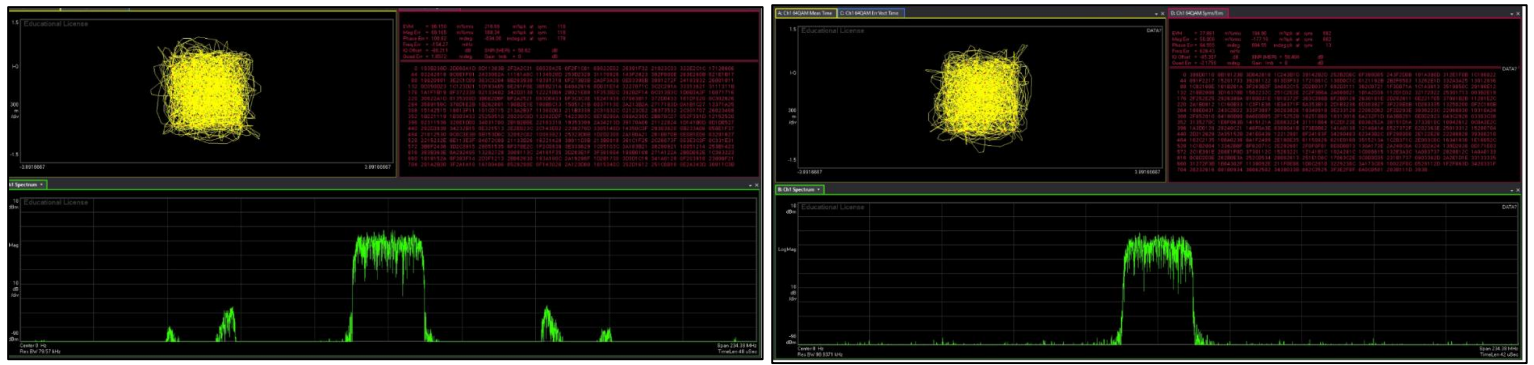


Figure 5 Signal Spectrum Before and After FIR Filtering for L=5

The spectrum comparison shows that the FIR filter attenuates the unwanted spectral images while preserving the required baseband signal.

Table 2: MATLAB Verification Results for the Floating- and Fixed-Point Models

Up-sampling Factor (L)	EVM [m%rms] Target: < 350m%rms	
	Floating Point	Fixed Point (Q3.12)
2	121	116.21
3	102	186.41
4	91	192.94
5	87	77.881

The MATLAB results demonstrate that the fixed-point implementation maintains the behavior of the floating-point MINAJ2 model. The FIR filter suppresses unwanted spectral images, and the final signal satisfies the EVM requirement. The fixed-point model was therefore used as the reference for RTL verification

4 ASIC Implementation

This chapter describes the hardware implementation of the interpolation ASIC, from the fixed-point algorithmic model to the final placed-and-routed design. The implementation flow included RTL development and functional verification, logic synthesis and scan insertion, logical equivalence checking, multi-corner static timing analysis, physical design, and post-layout verification. The ASIC was implemented using a TSMC 65 nm digital standard-cell process.

4.1 Design flow and tools

The project followed a standard RTL-to-GDSII digital implementation flow. The main tools and their roles are listed in Table 3.

Table 3: Design Tools and Platforms

Implementation Stage	Tool or Platform	Role in the Project
RTL development	SystemVerilog	Description of the control logic, I/Q datapaths, MINAJ2 interpolators, sample-shift stages, and programmable FIR filters
Simulation and debugging	Synopsys VCS and Verdi	RTL and gate-level simulation, waveform inspection, and functional debugging
Logic synthesis and DFT	Synopsys Design Compiler	RTL synthesis, technology mapping, timing and area optimization.
Logical equivalence checking	Cadence Conformal	Verification of equivalence between the RTL, synthesized, and scan-inserted netlists
Static timing analysis	Synopsys PrimeTime	Setup and hold analysis across the required process, voltage, and temperature corners
Physical implementation	Cadence Innovus	Floorplanning, power planning, placement, clock-tree synthesis, routing, and post-layout optimization
Power analysis	Cadence Innovus	Activity-based power estimation using switching activity obtained from gate-level simulations
Technology platform	TSMC 65 nm libraries	Standard-cell, I/O, timing, physical, and parasitic data used throughout the implementation flow
Layout Versus Schematic (LVS)	Siemens Calibre	Comparison of the extracted layout connectivity with the implemented circuit netlist

The flow began with functional RTL verification for all supported interpolation factors. The verified design was then synthesized, followed by logical equivalence and multi-corner timing checks. After physical implementation, the extracted design was evaluated using post-layout gate-level simulation, timing analysis, and activity-based power estimation.

4.2 System Requirements and Architecture

The ASIC receives serial I and Q sample streams at a fixed input sampling rate of 60 MSa/s and supports interpolation factors $L \in \{2,3,4,5\}$, resulting in an output sampling rate of $f_{out} = L \cdot 60$ MSa/s.

Two external clock configurations are used. For $L=2$ and $L=4$, the chip operates with a 960 MHz clock and signed 16-bit input words. For $L=3$ and $L=5$, a 900 MHz clock is used with signed 15-bit words, which are sign-extended internally to 16 bits. In both configurations, the serial interface maintains the same 60 MSa/s input-sample rate.

Table 4: ASIC Operating Modes

L	External Clock	Input Word Length	Input Rate	Output Rate
2	960 MHz	16 bits	60 MSa/s	120 MSa/s
3	900 MHz	15 bits	60 MSa/s	180 MSa/s
4	960 MHz	16 bits	60 MSa/s	240 MSa/s
5	900 MHz	15 bits	60 MSa/s	300 MSa/s

The top-level architecture, shown in Figure 6, consists of a shared control section and two parallel I/Q processing paths. Each path performs serial-to-parallel conversion, sample storage, MINAJ2 interpolation, sample sequencing, and programmable 10-tap FIR filtering. The I path acts as the timing master and shares the required word and interpolation-phase information with the Q path, maintaining synchronization between the two signal components.

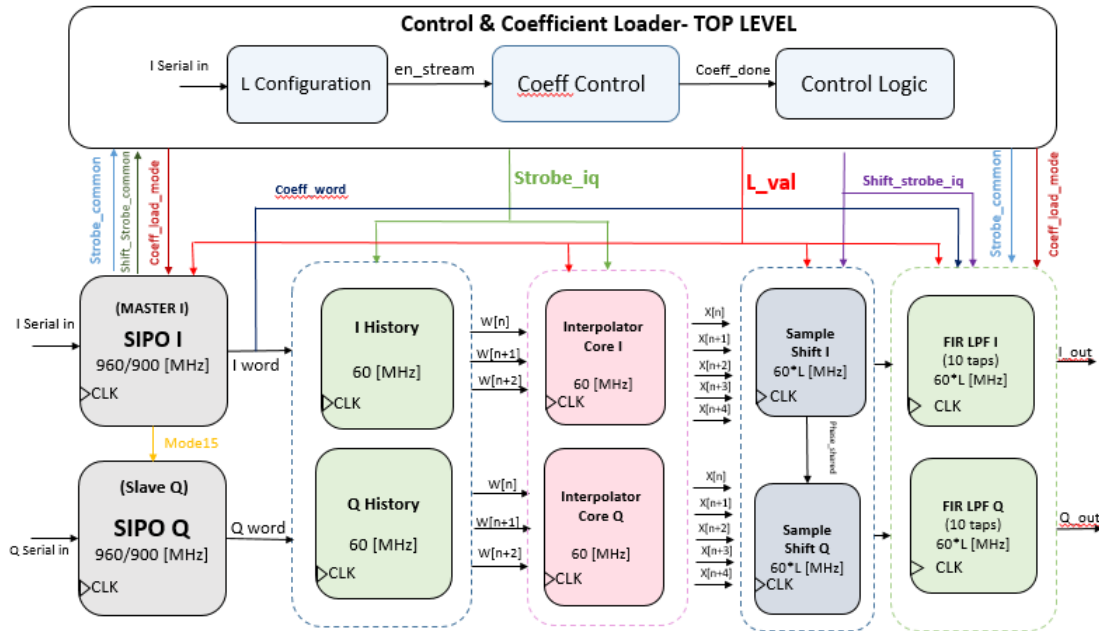


Figure 6: Top-Level Architecture of Interpolation ASIC

4.3 RTL Architecture and Implementation

The architecture described in Section 4.2 was implemented in System-Verilog using fixed-point arithmetic. The incoming serial sequence begins with a synchronization bit used to detect the start of configuration, followed by the three-bit interpolation factor L . The ten FIR coefficients are then loaded through the I serial input, and normal I/Q streaming begins only after configuration is complete. The same coefficient set is loaded into both FIR filters.

Since the different stages operate at different effective sample rates, strobe-based architecture was used rather than creating additional clock domains. All sequential logic is driven by the external 900 or 960 MHz clock, while strobes enable the processing stages at the required rates. strobe_iq is generated once for each complete I/Q input word and therefore operates at 60 MHz. It updates the sample-history registers and starts the processing of a new MINAJ2 interpolation

window. `shift_strobe_iq` operates at 60L MHz and transfers the generated interpolation samples sequentially toward the FIR filters.

The I channel acts as the timing master. It determines the serial-word boundaries and generates the interpolation timing, while the corresponding word mode and interpolation phase are shared with the Q channel. This ensures that the I and Q components remain synchronized throughout the datapath. The Sample Shift stage converts each group of L MINAJ2 results into a continuous 60L MSa/s stream, which is then processed by the programmable 10-tap FIR filters.

4.4 Verification

The RTL was verified using simulation-based functional verification, formal property verification, and coverage analysis. Verification was performed at both block and top levels to check arithmetic correctness, control behavior, I/Q synchronization, and operation for all supported interpolation factors.

4.4.1 Functional Verification

Self-checking SystemVerilog testbenches were developed for the main RTL blocks, including the configuration and serial interfaces, sample-history storage, MINAJ2 interpolation, Sample Shift, and FIR filtering. Directed tests, reset and boundary conditions, and constrained-random stimulus were used for block-level verification. The MINAJ2 core was also compared with a bit-accurate reference calculation for $L = \{2, 3, 4, 5\}$.

End-to-end verification was performed using the top-level `tb_corner` testbench together with the MATLAB fixed-point golden model (`run_all_modes.m`). The regression covers all four interpolation factors, all twelve ordered transitions between different L values, long 64-QAM sequences, and corner cases including zero, DC, impulse, full-scale, and saturation inputs. Additional testbench scenarios check reset during coefficient loading and I/Q streaming, repeated reset, and illegal configuration headers. The RTL I/Q outputs are aligned and compared sample-by-sample with the MATLAB golden output.

4.4.2 Formal Verification

Block-level formal verification was performed using Synopsys VC Formal in FPV mode. Formal checkers were applied to the configuration logic, serial I/Q interfaces, sample-history and interpolation-result registers, I/Q Sample Shift blocks, MINAJ2 core, and FIR filter.

The properties verify key behaviors including configuration and word timing, state update and hold conditions, interpolation-phase progression, sample shifting, coefficient loading, and MINAJ2 arithmetic saturation. Assumptions were used where necessary to constrain each block to valid system-level operating conditions.

4.4.3 Coverage

Simulation coverage was used to verify that the relevant operating modes and corner conditions were exercised, including all supported L values, mode transitions, interpolation phases, control conditions, and arithmetic saturation cases. Formal cover properties were also used to confirm

the reachability of important states, such as each interpolation mode and the maximum valid phase for each L.

The final functional, coverage, and formal verification results are presented in Chapter 6.

4.5 Synthesis

Following RTL verification, `top_interpolator_dac` was synthesized using Synopsys Design Compiler W-2024.09-SP3 with `compile_ultra` and mapped to TSMC 65 nm low-power RVT, LVT, and HVT cells.

4.5.1 Timing Constraints

The SDC specified a 1.04167 ns clock period (960 MHz), 0.05 ns clock uncertainty, 0.20 ns maximum transition, and 0.10 ns I/O delays. Multicycle constraints reflected the strobe-controlled updates: 15/14 setup/hold cycles for the MINAJ2 and interpolation-window registers and 3/2 cycles for the FIR outputs, allowing these paths additional clock cycles to complete according to their functional update rates and preventing unnecessary timing violations.

4.5.2 Logic Optimization and Clock Gating

Latch-based clock gating reduced switching in enable-controlled registers. The MINAJ2, FIR, and Sample Shift hierarchies were preserved for implementation and equivalence checking.

4.5.3 Synthesis Results

The final mapped design contained 25,916 cells, including 1,063 sequential cells. The worst setup slack was 0.00 ns (MET), while a small pre-layout hold violation of approximately -0.03 ns remained on short input-to-register paths and was checked again after physical implementation. The main outputs were the mapped gate-level netlist, SDC constraints, Design Compiler databases, and files required for LEC, GLS, STA, and physical implementation.

4.6 Post-Synthesis Validation

After synthesis, the design was validated using Logic Equivalence Checking (LEC) and Gate-Level Simulation (GLS).

4.6.1 Logic Equivalence Checking

Cadence Conformal LEC compared the RTL with the synthesized gate-level netlist. The comparison completed with a PASS result, with 0 incomplete verification points and 0 design ambiguities, confirming that synthesis preserved the RTL functionality.

4.6.2 Pad Integration and GLS

The top wrapper connected `top_interpolator_dac` to the TSMC 65 nm I/O and supply pads. Pad connectivity and placement were defined in `top.v` and `top.io`, respectively. Pad-level GLS in VCS verified configuration, coefficient loading, and I/Q streaming using the I/O cell models. The verified design data were then transferred to Innovus for physical implementation.

5 Physical Design

The pad-integrated design was physically implemented in **TSMC 65 nm technology** using Cadence Innovus. The implementation flow included design initialization, floorplanning, power-grid generation, placement, Clock Tree Synthesis (CTS), routing, post-route optimization, parasitic extraction, and physical verification. Final post-layout timing sign-off was performed using **Synopsys PrimeTime**.

5.1 Design Initialization

The pad-level synthesized netlist was imported into Innovus together with the technology LEF files, standard-cell and I/O libraries, pad locations, timing constraints, and multi-corner analysis setup. Slow, typical, and fast operating conditions were defined to support timing analysis throughout the implementation flow.

The clock and timing constraints defined during synthesis were retained during physical implementation.

5.2 Floorplanning and Power Grid

The floorplan defined the core region inside the I/O pad ring while reserving sufficient routing resources for signal and power distribution. Power and ground rings and stripes were generated and connected to the standard-cell rails and supply pads.

The final die dimensions are **1000 × 950 μm^2** , with a core area of **385,016 μm^2** . I/O filler cells were inserted to complete the pad ring and maintain power continuity.

5.3 Placement and Clock Tree Synthesis

Standard cells were placed and optimized considering timing and routing congestion. Tie-high and tie-low cells were inserted where required, followed by legalization and placement optimization.

Clock Tree Synthesis was performed using **Cadence CCOpt**. Clock buffers were inserted and optimized to distribute the main clock to the sequential and clock-gating cells while controlling skew, insertion delay, loading, and transition time.

5.4 Routing and Design Closure

Following CTS, detailed routing was performed to complete the signal and power interconnects. Post-route optimization and ECO corrections were then applied to improve timing and resolve routing and electrical issues.

Placement legality, routing connectivity, timing, and physical-rule checks were repeated throughout design closure. Physical filler cells were inserted before extraction and final sign-off verification.

5.5 Parasitic Extraction

After routing, interconnect resistance and capacitance were extracted for the required RC conditions. The final post-layout gate-level netlist and extracted SPEF files were used for sign-off analysis.

Synopsys PrimeTime used the extracted parasitics together with the timing libraries and SDC constraints for final static timing analysis.

5.6 Post-Layout Static Timing Analysis

Final Static Timing Analysis was performed using **Synopsys PrimeTime** with the post-layout netlist, extracted parasitics, timing libraries, and timing constraints. Setup and hold timing were evaluated under slow and fast operating conditions.

OCV derating and propagated clock paths were included in the final timing analysis. The detailed PrimeTime reports and scripts are provided in the project repository.

5.7 Post-Route Power Analysis

Post-route power analysis was performed using SAIF switching activity obtained from gate-level simulations for interpolation factors $L \in \{2,3,4,5\}$. The switching activity was annotated onto the implemented pad-level design and used to evaluate internal, switching, leakage, and total power. Power was evaluated under Slow, Typical, and Fast operating conditions. The detailed numerical results are presented in Chapter 6.4.

5.8 Physical Verification

Physical verification was performed on the final routed layout using **Calibre**. Design Rule Checking (DRC) was used to verify compliance with the TSMC 65 nm physical-design rules, while Layout Versus Schematic (LVS) was used to compare the extracted layout connectivity with the implemented circuit netlist.

The remaining DRC markers were limited to ESD-related and density/dummy-fill rules, which were identified as accepted foundry-level exceptions. The complete physical-verification reports are provided in the project repository.

6 Results and Analysis

6.1. RTL vs. MATLAB EVM

Table 5: EVM Comparison Between the MATLAB Reference Model and RTL Implementation

Up-sampling factor (L)	MATLAB EVM [m%rms]	RTL EVM [m%rms]	Relative EVM difference [%]	Requirement Target: < 350m%rms
2	116.21	110.00	5.34%	Pass
3	186.41	174.88	6.19%	Pass
4	192.94	186.37	3.41%	Pass
5	77.88	72.80	6.52%	Pass

Table 5 compares the EVM obtained from the MATLAB reference model with the corresponding RTL results. Both models satisfy the required EVM limit for every interpolation factor. The RTL results closely follow the MATLAB reference, with a maximum absolute difference of 11.53m%rms for L=3. The small deviations are attributed to fixed-point rounding, truncation, and signal-alignment differences. The lowest EVM was obtained for L=5, while the highest value, obtained for L=4, remained well below the specified limit. These results confirm the correct implementation of the MINAJ2 interpolator and FIR filter in RTL.

6.2 Verification Results

6.2.1 Functional Simulation Results

Top-level RTL verification was performed using the complete regression environment described in Section 4.4.1. The regression covered all supported interpolation factors $L=\{2,3,4,5\}$, all twelve ordered transitions between different L values, 64-QAM input sequences, arithmetic corner cases, reset scenarios, and illegal configuration cases.

A representative top-level waveform is shown in Figure 7. It demonstrates the relationship between the input-rate `strobe_iq`, the output-rate `shift_strobe_iq`, and the interpolation phase during normal operation for L=4 and L=5.

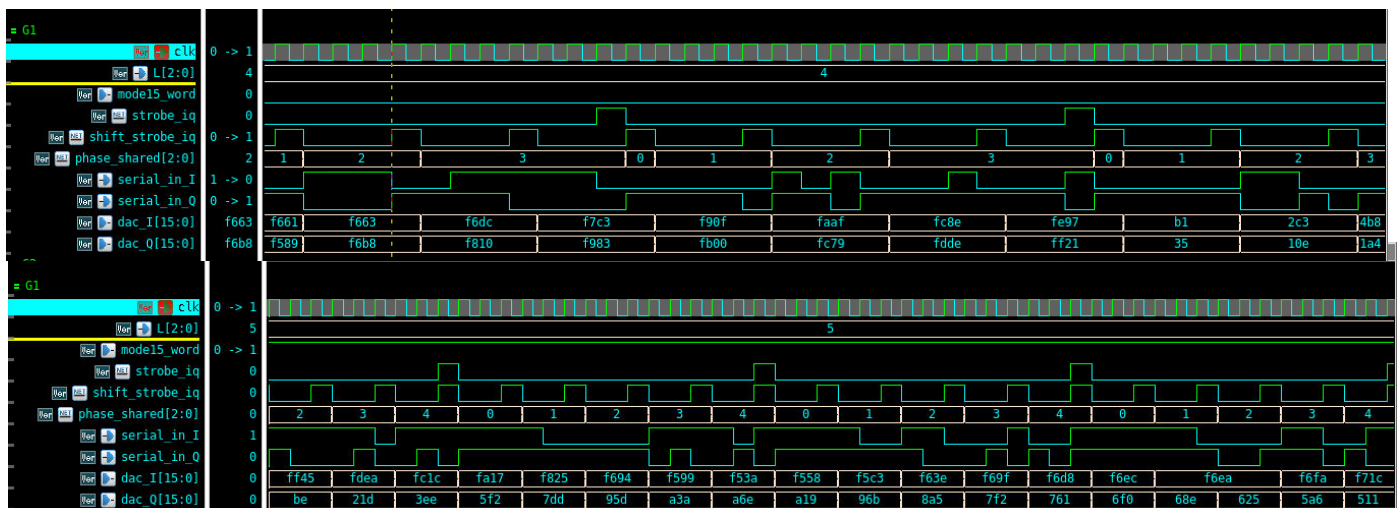


Figure 7: Representative RTL Timing and Interpolation Strobes

The RTL output was aligned and compared sample by sample with the MATLAB fixed-point golden model. Table 6 summarizes representative comparisons of the four supported interpolation factors. After applying the reported alignment offset, all I and Q samples matched the golden model exactly.

Table 6: Representative Bit-Exact RTL-to-MATLAB Comparison

<i>L</i>	MATLAB samples	RTL samples before alignment	Best offset	I/Q mismatches	Result
2	24,000	24,125	6	0	Pass
3	36,000	36,200	8	0	Pass
4	48,000	48,250	10	0	Pass
5	60,000	60,333	12	0	Pass

The zero-mismatch results confirm that the RTL implementation reproduces the bit-accurate behavior of the MATLAB fixed-point model for all supported interpolation factors. The complete regression also passed for all tested mode transitions, reset conditions, arithmetic corner cases, and illegal configuration scenarios.

6.2.2 Coverage Results

Functional and assertion coverage were collected during RTL simulation. The defined coverage goals for the supported interpolation modes, mode transitions, interpolation phases, control behavior, and arithmetic corner cases were exercised. Both `spline_cov.sv` and `spline_sva.sv` achieved 100% coverage.

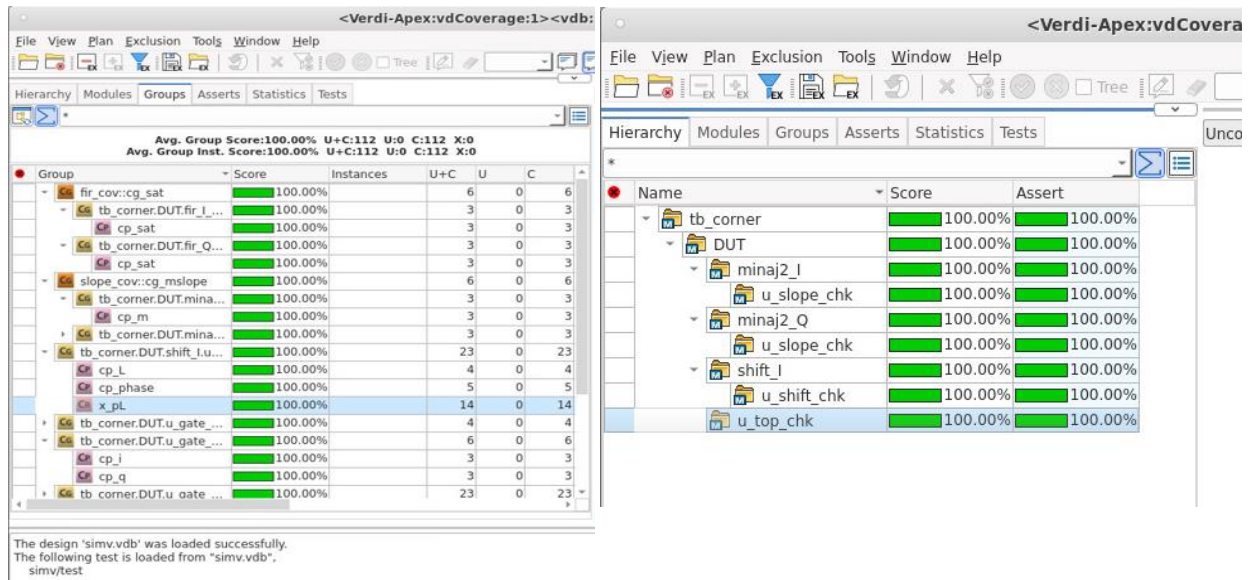


Figure 8: Final RTL Functional and Assertion Coverage Results

6.2.3 Formal Verification Results

Block-level formal verification was performed using Synopsys VC Formal on nine major RTL blocks. A total of 123 assertions were analyzed, of which 118 were proven and five were classified as vacuous, with no failed assertions. In addition, all 38 cover properties were reached, achieving

100% cover-property completion. The five vacuous properties were reset-related assertions in runs where reset was constrained inactive. The complete formal verification reports are available in the project documentation on GitHub.

6.3 Physical-Design Results

The final layout occupied a **1000 × 950 μm^2 die**, corresponding to a total die area of **0.950 mm^2** . The core area was **0.385 mm^2** , while the logical standard-cell area excluding physical cells was approximately **0.149 mm^2** . This corresponds to a logical core utilization of **38.77%**.

The final implementation contained approximately **138,218 equivalent gates**.

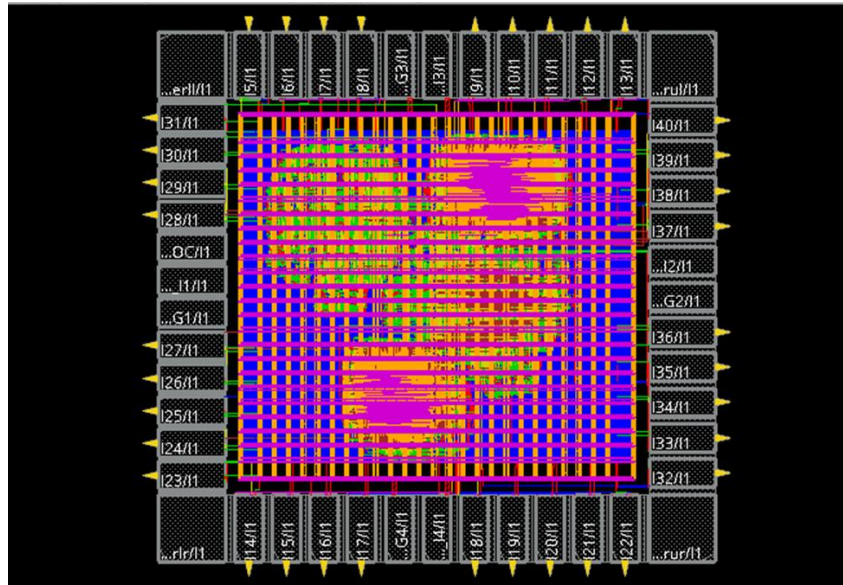


Figure 9: Final Routed Pad-Level Layout in Cadence Innovus

Table 7: Final Physical-Design Results

Parameter	Final Result
Technology	TSMC 65 nm
Die dimensions	1000 × 950 μm^2
Die area	0.950 mm^2
Core area	0.385 mm^2
Logical cell area	0.149 mm^2
Logical core utilization	38.77%
Equivalent gate count	138,218

6.4 Post-Route Power Results

Post-layout power was evaluated for all interpolation factors under Slow, Typical, and Fast operating conditions.

Table 8: Post-Route Power Results

L	Slow Core [mW]	Slow Total [mW]	Typical Core [mW]	Typical Total [mW]	Fast Core [mW]	Fast Total [mW]
2	16.32	93.20	19.53	80.70	24.24	128.53
3	20.78	112.57	25.04	103.60	30.84	155.43
4	24.47	115.26	29.62	107.01	36.27	159.50
5	28.75	117.16	34.89	109.51	42.63	162.62

Power consumption increased with the interpolation factor due to the higher switching activity at larger output rates. The highest core power was **42.63 mW** at L=5 under the Fast condition. The total chip power also includes the I/O pads and other pad-level circuitry.

6.5 IR Drop

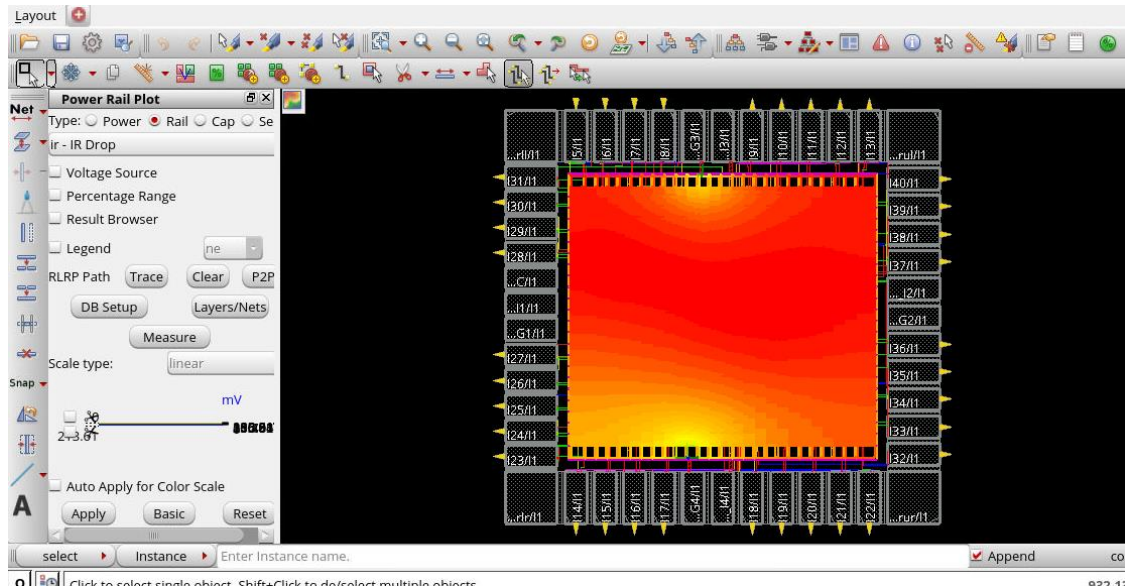


Figure 10: Static IR-Drop Analysis of the Final Layout

Static IR-drop analysis under the Slow view showed clean VDD/VSS connectivity, with 0 violations and 0 warnings. The worst-case VDD drop was about 189.1 mV, while the maximum VSS ground rise was about 213.6 mV. The results indicate a connected power grid, but with areas that could be further strengthened to reduce voltage drop.

6.6 PrimeTime Timing Sign-Off

Final timing sign-off was performed using Synopsys PrimeTime after routing and parasitic extraction. Setup and hold checks were performed under slow, typical, and fast operating conditions.

No negative-slack setup or hold paths were reported in any of the analyzed conditions, confirming post-layout timing closure. The worst reported setup and hold paths remained positive, with **+0.29 ns setup slack** and **+0.86 ns hold slack**, respectively.

Table 9: PrimeTime Timing Sign-Off Results

Timing Check	Worst Slack	Violating Paths	Result
Setup	+0.29 ns	0	Pass
Hold	+0.86 ns	0	Pass

6.7 DRC and LVS

Final physical verification was performed using Calibre on the completed GDSII layout.

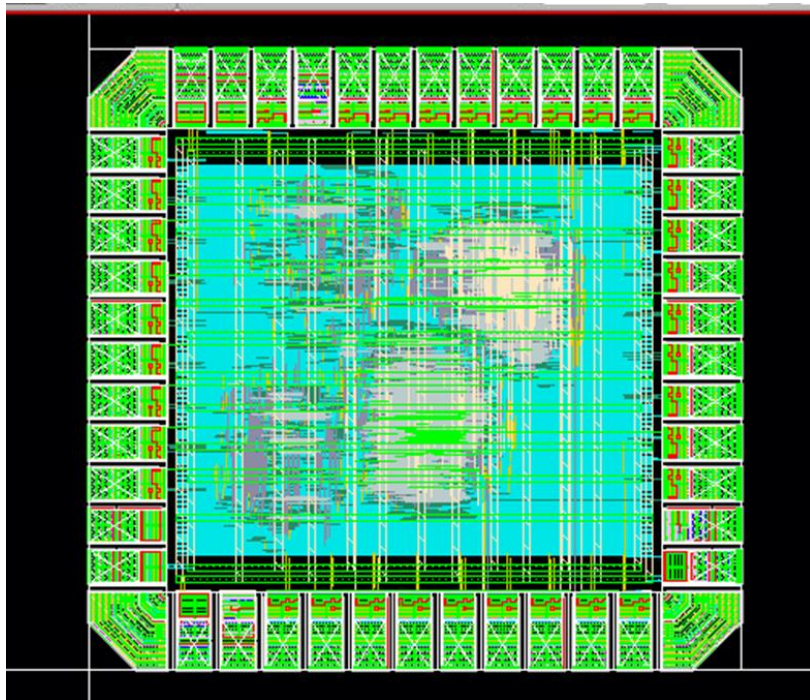


Figure 11: Final GDSII Layout Viewed in Cadence Virtuoso

The remaining DRC markers were limited to **ESD-related and density/dummy-fill rules**, which were identified as accepted foundry-level exceptions. LVS was used to verify the correspondence between the extracted physical layout and the implemented circuit netlist.

The complete DRC and LVS reports are provided in the project repository.

6.8 Final Sign-Off Summary

Table 10: Final Sign-Off Summary

Category	Final Result	Status
Functional Verification	All $L \in \{2,3,4,5\}$ operating modes verified	Pass
Signal Accuracy	EVM requirement satisfied	Pass
PrimeTime Setup	Worst slack = +0.29 ns, 0 violations	Pass
PrimeTime Hold	Worst slack = +0.86 ns, 0 violations	Pass
Core Area	0.385 mm ²	—
Die Area	0.950 mm ²	—
Gate Count	138,218 equivalent gates	—
Logical Core Utilization	38.77%	—
Maximum Core Power	42.63 mW	—
Maximum Total Chip Power	162.62 mW	—
DRC	Accepted ESD and density-related exceptions only	Accepted
LVS	Layout-to-netlist verification performed	Pass

The final implementation satisfied the main functional, signal-quality, and timing objectives of the project. All four interpolation modes were successfully implemented, and post-layout PrimeTime analysis achieved setup and hold timing closure across the analyzed operating conditions.

The final layout occupied a **0.950 mm² die** with a **0.385 mm² core**. Power increased with interpolation factor as expected due to the higher switching activity at higher output rates. Physical verification showed only the accepted ESD and density-related DRC exceptions

7. Summary, Conclusions, and Future Work

7.1 Evaluation Against the Project Objectives

The project achieved its main objectives:

- The system supports $L \in \{2,3,4,5\}$, producing output sampling rates of 120–300 MS/s.
- A synchronized fixed-point architecture was implemented for the parallel I and Q datapaths.
- The MINAJ2 interpolator and configurable 10-tap FIR filter met the required signal-quality targets.
- RTL, gate-level, and pad-level verification showed consistent functionality, while LEC and formal verification provided additional confidence in correctness.
- Final PrimeTime analysis achieved timing closure, with worst setup and hold slacks of **+0.29 ns** and **+0.86 ns**.
- The final design occupied a **0.950 mm² die** with a **0.385 mm² core**, **38.77% logical utilization**, and approximately **138k equivalent gates**.
- Post-layout core power ranged from **16.32 to 42.63 mW**, while total chip power ranged from **80.70 to 162.62 mW** across the analyzed operating conditions.
- Final physical verification was completed, with only accepted ESD and density-related DRC exceptions remaining.

The project did not include chip fabrication or laboratory measurements. Therefore, the final conclusions are based on simulation, formal verification, and post-layout sign-off analysis.

7.2 Conclusions

This project demonstrated that cubic MINAJ2 interpolation can be implemented efficiently in fixed-point hardware while maintaining the required 64-QAM signal quality. The configurable architecture supports interpolation factors $L=2,3,4,5$ using parallel I/Q datapaths, shared control, and programmable FIR filtering.

The design was validated throughout the complete development flow. MATLAB floating- and fixed-point models established the numerical reference, RTL simulation verified functional behavior, and formal verification and LEC were used to confirm logical correctness. The synthesized design was then taken through physical implementation, followed by post-layout timing, power, and physical-verification analysis.

Overall, the project successfully completed the flow from algorithm development and fixed-point modeling to a routed TSMC 65 nm ASIC implementation. The final design met the main functional, signal-quality, timing, and implementation objectives of the project.

7.3 Suggested Performance Improvements

Possible improvements to the current design include:

- Further optimization of the FIR coefficients and fixed-point word lengths.
- Evaluation of a polyphase FIR architecture to reduce switching activity and power.
- Additional clock-gating and arithmetic optimizations to reduce dynamic power.
- Optimization of the I/O-pad configuration to reduce total chip power.
- Strengthening of the power-distribution network to reduce IR drop.

7.4 Future Work

Future work could include:

- ASIC fabrication and laboratory characterization of EVM, power consumption, and maximum operating frequency.
- FPGA prototyping of the same MINAJ2 interpolation chain for real-time hardware validation before fabrication.
- Support for additional interpolation factors and output sampling rates.
- Extension of the programmable FIR to support additional coefficient sets and filter responses.
- Power-delivery optimization, followed by dynamic IR-drop and electro-migration analysis.
- Addition of scan insertion and ATPG for manufacturing-test generation and fault-coverage evaluation.
- Evaluation of alternative MINAJ2 hardware architectures to study area, power, timing, and implementation trade-offs.

These extensions would provide a path toward silicon validation and integration into a complete communication system

8 Project Documentation

The complete project is documented in a public GitHub repository containing the source code, verification environments, implementation scripts, selected reports, and technical documentation used throughout the ASIC flow. The repository follows the same organization as the project development stages, from MATLAB modeling and RTL design through synthesis, physical implementation, and signoff.

Table 11: Final GitHub repository organization.

Folder	Contents
MATLAB/	64-QAM signal generation, floating- and fixed-point MINAJ2 models, FIR coefficient design, RTL golden model, EVM analysis, and plots
RTL/	Final synthesizable SystemVerilog implementation of the interpolation core
Verification/	Block-level and top-level testbenches, SystemVerilog Assertions, functional coverage, regression tests, and formal-verification files
Synthesis/	Design Compiler synthesis scripts, timing constraints, generated netlist, and selected synthesis reports
saif_generation/	Testbench, mode-specific input vectors, and scripts used to generate SAIF switching activity for L=2,3,4,5
Innovus/	Physical-design scripts for floorplanning, power planning, placement, CTS, routing, ECO, extraction, and post-route power analysis
Signoff/	RTL-to-gate LEC, pad-level GLS, PrimeTime timing analysis, Calibre DRC, and Calibre LVS
Figures/	Architecture diagrams, final layout image, and figures used in the documentation
docs/	Detailed documentation of the architecture, MATLAB model, RTL implementation, verification, synthesis, physical design, power analysis, signoff, and final results
README.md	Project overview, main specifications, design flow, repository structure, and links to the detailed documentation

The docs/ directory provide a detailed description of each design stage, while the corresponding implementation folders contain the scripts, source files, and selected reports required to support the presented results. This structure allows the complete RTL-to-layout methodology to be followed while keeping the repository focused on the final project flow.

Large generated simulation databases and temporary EDA files are excluded from version control. Proprietary PDK data, technology libraries, foundry models, license files, and other restricted technology information are also not included in the public repository.

Repository: https://github.com/marianajjar/Spline_ASIC

References

- [1] R. Dębski, “Real-time interpolation of streaming data,” *Computer Science*, vol. 21, no. 4, pp. 513–532, 2020, doi: 10.7494/csci.2020.21.4.3932.
- [2] A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ, USA: Pearson, 2010.
- [3] J. G. Proakis and M. Salehi, *Digital Communications*, 5th ed. New York, NY, USA: McGraw-Hill, 2008.
- [4] MathWorks, “firpm — Parks-McClellan optimal FIR filter design,” *MATLAB Signal Processing Toolbox Documentation*.
- [5] MathWorks, *Fixed-Point Designer Documentation*, MathWorks, Natick, MA, USA.
- [6] IEEE, *IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language*, IEEE Std 1800-2023, 2023.
- [7] G. Samuel, *Backend Manual—TSMC 65 nm*. Haifa, Israel: Technion—Israel Institute of Technology, Andrew and Erna Viterbi Faculty of Electrical and Computer Engineering, VLSI Laboratory, Jul. 18, 2024. Based on a manual by A. Gorbos, E. Gorbos, L. Karkaby, and S. Abu Ghanima.
- [8] Synopsys, *Design Compiler User Guide*, Release X-2025.06-SP4, Synopsys licensed documentation, 2025.
- [9] Synopsys, *VCS User Guide*, Synopsys licensed documentation.
- [10] Synopsys, *VC Formal User Guide*, Synopsys licensed documentation.
- [11] Synopsys, *PrimeTime User Guide*, Synopsys licensed documentation.
- [12] Cadence Design Systems, *Innovus Implementation System User Guide*, Cadence licensed documentation.
- [13] Cadence Design Systems, *Conformal Equivalence Checking User Guide*, Cadence licensed documentation.
- [14] Siemens EDA, *Calibre Design Verification User’s Manual*, Siemens Digital Industries Software, licensed documentation.
- [15] Taiwan Semiconductor Manufacturing Company (TSMC), *TSMC 65 nm Standard-Cell Library Documentation*, proprietary technology-library documentation.
- [16] Taiwan Semiconductor Manufacturing Company (TSMC), *TSMC 65 nm I/O Library Documentation*, proprietary technology-library documentation.
- [17] Keysight Technologies, *89600 VSA Software Documentation*, Keysight Technologies.